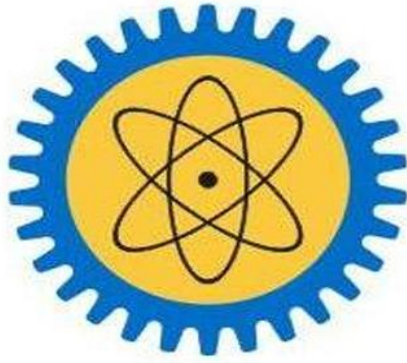




**BUSITEMA
UNIVERSITY**
Pursuing Excellence

**MATLAB ASSIGNMENT
GROUP 13
WAR 2**

NAME	REG NO
KYESIMIRA THEODORE	BU/UG/2025/4441
ATUKWATSE BRUCE	BU/UP/2025/2108
AINEMBABAZI MERCY	BU/UP/2025/3248
MUWAYIRE GASTER	BU/UG/2025/4444
AKWIR MERCY LEAH	BU/UP/2025/1532

1. Newton-Raphson Method

Problem Statement

Find the root of the equation $f(x) = x^2 - 4 = 0$ using the Newton-Raphson method, starting from an initial guess $x_0 = 3$, correct to a tolerance of 0.0001.

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

Here $f(x) = x^2 - 4$ and $f'(x) = 2x$. The exact root is $x = 2$.

Pseudo code

1. Start
2. Read initial guess x_0 , tolerance e , and maximum iterations
3. Define $f(x)$ and $f'(x)$
4. Repeat: compute $x_1 = x_0 - f(x_0)/f'(x_0)$
5. Compute the error = $|x_1 - x_0|$
6. If $|x_1 - x_0| < \text{error}$, the root has been found: display x_1 and stop
7. Otherwise, set $x_0 = x_1$ and repeat from step 4
8. Stop

Flowchart

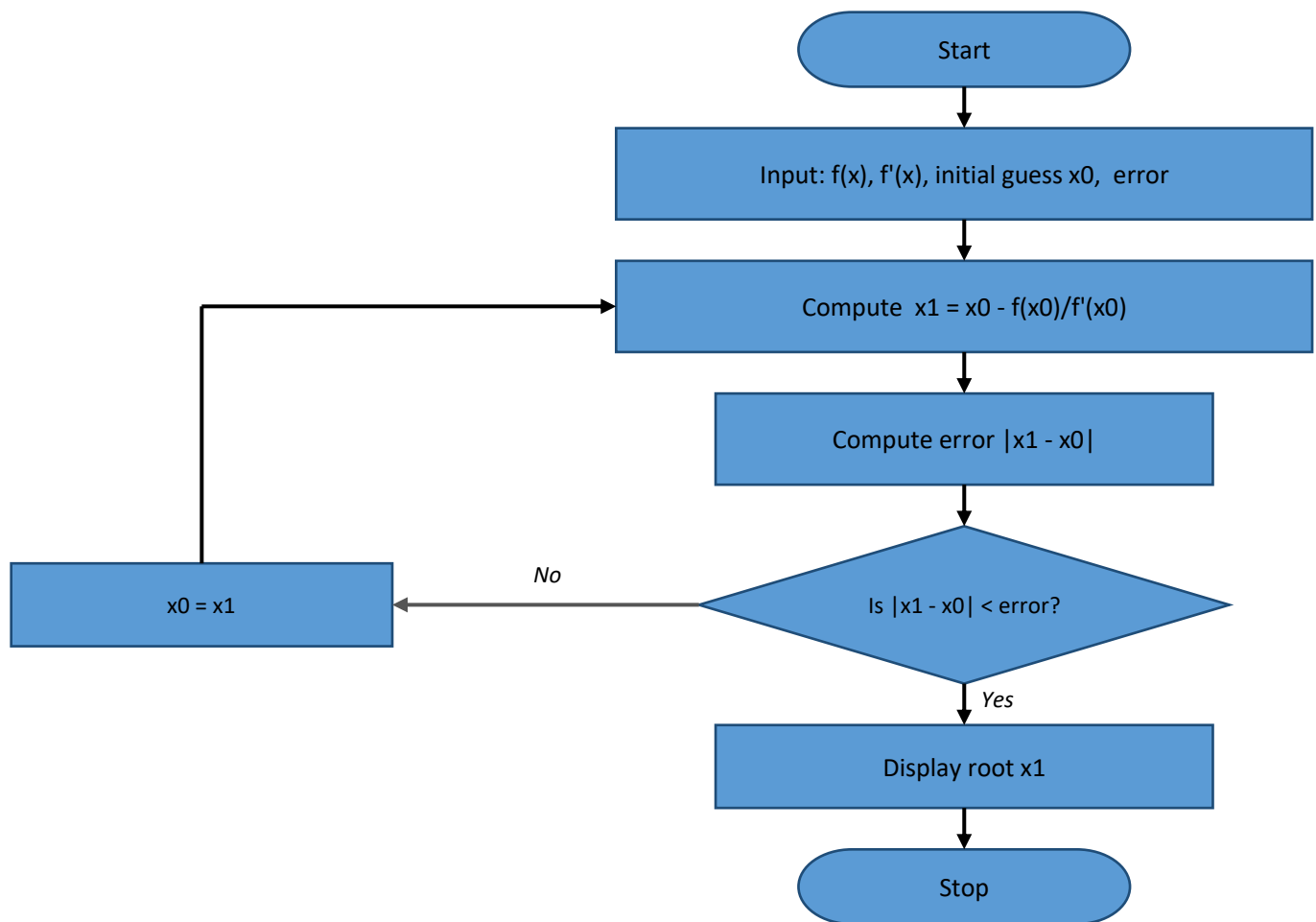


Figure 1: Flowchart of the Newton-Raphson Method

MATLAB Code - "while" loop version

This version repeats the update step until the change between successive approximations is smaller than the tolerance.

```

clc; clear;
f = @(x) x^2 - 4;      % function whose root we want
df = @(x) 2*x;         % derivative of f(x)

x0 = 3;                % initial guess
tol = 1e-4;            % tolerance
maxIter = 50;          % safety limit

iter = 0;
err = 1;               % force entry into the loop

while err > tol && iter < maxIter
    x1 = x0 - f(x0)/df(x0);
    err = abs(x1 - x0);
    x0 = x1;
    iter = iter + 1;
    fprintf('Iteration %d: x = %.6f\n', iter, x0);
end

fprintf('Root = %.6f found in %d iterations\n', x0, iter);
  
```

Sample Output:

Iteration 1: x = 2.166667

Iteration 2: x = 2.006410

Iteration 3: x = 2.000010

Iteration 4: x = 2.000000

Root = 2.000000 found in 4 iterations

MATLAB Code - "for" loop version

This version runs a fixed number of iterations, useful when the number of steps is decided in advance rather than by a tolerance check.

```
clc; clear;
f = @(x) x^2 - 4;
df = @(x) 2*x;

x0 = 3;           % initial guess
n = 6;           % fixed number of iterations

for iter = 1:n
    x1 = x0 - f(x0)/df(x0);
    fprintf('Iteration %d: x = %.6f\n', iter, x1);
    x0 = x1;
end

fprintf('Approximate root after %d iterations = %.6f\n', n, x0);
```

Sample Output:

Iteration 1: x = 2.166667

Iteration 2: x = 2.006410

Iteration 3: x = 2.000010

Iteration 4: x = 2.000000

Iteration 5: x = 2.000000

Iteration 6: x = 2.000000

Approximate root after 6 iterations = 2.000000

More Examples

Example 1: $f(x) = x^2 - 4$, $x_0 = 3$.

$$f'(x) = 2x.$$

Iteration 1:

$$f(3) = 3^2 - 4 = 5$$

$$f'(3) = 2(3) = 6$$

$$x_1 = 3 - 5/6 = 2.166667 \quad \text{error} = |2.166667 - 3| = 0.833333$$

Iteration 2:

$$f(2.166667) = 0.694444,$$

$$f'(2.166667) = 4.333333$$

$$x_2 = 2.166667 - 0.694444/4.333333 = 2.006410$$

All iterations:

n	X_n	$f(X_n)$	$f'(X_n)$	X_{n+1}	Error
1	3.000000	5.000000	6.000000	2.166667	0.833333
2	2.166667	0.694444	4.333333	2.006410	0.160256
3	2.006410	0.025682	4.012821	2.000010	0.006400
4	2.000010	0.000041	4.000020	2.000000	0.000010

The error is below 0.0001 after iteration 4, so the root is $x \approx 2.000000$ (exact root: 2).

Example 2: Given that $f(x) = x^2 - 2$, find $\sqrt{2}$ where $x_0 = 1$.

solution

$$f(x) = x^2 - 2$$

$$f'(x) = 2x.$$

n	X_n	$f(X_n)$	$f'(X_n)$	X_{n+1}	Error
1	1.000000	-1.000000	2.000000	1.500000	0.500000
2	1.500000	0.250000	3.000000	1.416667	0.083333
3	1.416667	0.006944	2.833333	1.414216	0.002451

After 3 iterations $x \approx 1.414216$, which is within 0.000002 of $\sqrt{2} = 1.414214$.

2. Secant Method

Problem Statement

Find the root of the equation $f(x) = x^3 - x - 2 = 0$ using the Secant method, starting from two initial guesses $x_0 = 1$ and $x_1 = 2$, correct to a tolerance of 0.0001.

$$x_2 = x_1 - f(x_1)(x_1 - x_0) / (f(x_1) - f(x_0))$$

The exact root (to 6 decimal places) is $x \approx 1.521380$.

Pseudo code

1. Start
2. Read two initial guesses x_0 and x_1 , and tolerance e
3. Define $f(x)$
4. Repeat: compute $x_2 = x_1 - f(x_1)(x_1 - x_0) / (f(x_1) - f(x_0))$
5. Compute the error = $|x_2 - x_1|$
6. If $|x_2 - x_1| < \text{error}$, the root has been found: display x_2 and stop
7. Otherwise, set $x_0 = x_1$, $x_1 = x_2$, and repeat from step 4
8. Stop

Flowchart

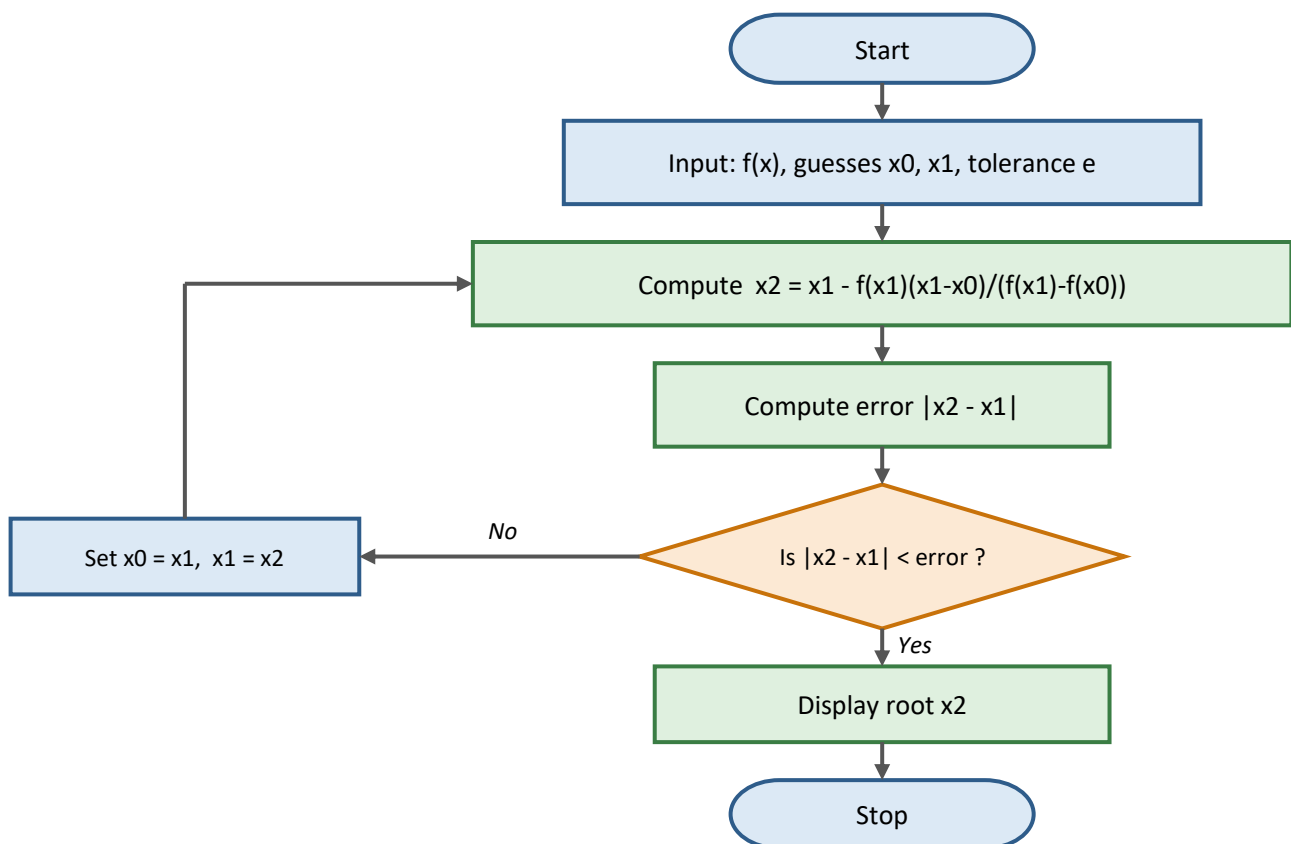


Figure 2: Flowchart of the Secant Method

MATLAB Code - "while" loop version

```
clc; clear;
f = @(x) x^3 - x - 2; % function whose root we want

x0 = 1; x1 = 2; % two initial guesses
tol = 1e-4;
maxIter = 50;

iter = 0;
err = 1;

while err > tol && iter < maxIter
    x2 = x1 - f(x1)*(x1 - x0)/(f(x1) - f(x0));
    err = abs(x2 - x1);
    x0 = x1;
    x1 = x2;
    iter = iter + 1;
    fprintf('Iteration %d: x = %.6f\n', iter, x1);
end

fprintf('Root = %.6f found in %d iterations\n', x1, iter);
```

Sample Output:

Iteration 1: x = 1.333333

Iteration 2: x = 1.462687

Iteration 3: x = 1.531169

Iteration 4: x = 1.520926

Iteration 5: x = 1.521376

Iteration 6: x = 1.521380

Root = 1.521380 found in 6 iterations

MATLAB Code - "for" loop version

```
clc; clear;
f = @(x) x^3 - x - 2;

x0 = 1; x1 = 2;
n = 6; % fixed number of iterations

for iter = 1:n
    x2 = x1 - f(x1)*(x1 - x0)/(f(x1) - f(x0));
    fprintf('Iteration %d: x = %.6f\n', iter, x2);
    x0 = x1;
    x1 = x2;
end

fprintf('Approximate root after %d iterations = %.6f\n', n, x1);
```

Sample Output:

Iteration 1: $x = 1.333333$

Iteration 2: $x = 1.462687$

Iteration 3: $x = 1.531169$

Iteration 4: $x = 1.520926$

Iteration 5: $x = 1.521376$

Iteration 6: $x = 1.521380$

Approximate root after 6 iterations = 1.521380

More Examples:

Example 1: $f(x) = x^3 - x - 2$, $x_0 = 1$, $x_1 = 2$.

Iteration 1:

$$f(1) = 1^3 - 1 - 2 = -2$$

$$f(2) = 8 - 2 - 2 = 4$$

$$x_2 = 2 - \frac{4(2 - 1)}{(4 - (-2))}$$

$$= 2 - \frac{4}{6} = 1.333333$$

$$\text{error} = |1.333333 - 2| = 0.666667$$

Iteration 2:

$$x_0 = 2,$$

$$x_1 = 1.333333,$$

$$f(x_0) = 4,$$

$$f(x_1) = -0.962963$$

$$x_3 = 1.333333 - \frac{(-0.962963)(1.333333 - 2)}{(-0.962963 - 4)}$$

$$= 1.462687$$

All iterations:

n	Xo	X1	f(X0)	f(X1)	X2	Error
---	----	----	-------	-------	----	-------

1	1.000000	2.000000	-2.000000	4.000000	1.333333	0.666667
2	2.000000	1.333333	4.000000	-0.962963	1.462687	0.129353
3	1.333333	1.462687	-0.962963	-0.333339	1.531169	0.068483
4	1.462687	1.531169	-0.333339	0.058626	1.520926	0.010243
5	1.531169	1.520926	0.058626	-0.002693	1.521376	0.000450
6	1.520926	1.521376	-0.002693	-0.000020	1.521380	0.000003

The error falls below 0.0001 at iteration 6, so the root is $x \approx 1.521380$.

Example 2: $f(x) = x^2 - 2$, $x_0 = 1$, $x_1 = 2$.

n	x0	X1	f(X0)	f(X1)	x2	Error
1	1.000000	2.000000	-1.000000	2.000000	1.333333	0.666667
2	2.000000	1.333333	2.000000	-0.222222	1.400000	0.066667
3	1.333333	1.400000	-0.222222	-0.040000	1.414634	0.014634
4	1.400000	1.414634	-0.040000	0.001190	1.414211	0.000423

After 4 iterations $x \approx 1.414211$, close to $\sqrt{2} = 1.414214$.

3. Runge-Kutta Method (4th Order)

Problem Statement

Solve the ordinary differential equation $dy/dx = x + y$, with initial condition $y(0) = 1$, using the fourth-order Runge-Kutta method with step size $h = 0.1$, to find y at $x = 0.4$.

```
k1 = h f(x, y)
k2 = h f(x + h/2, y + k1/2)
k3 = h f(x + h/2, y + k2/2)
k4 = h f(x + h, y + k3)
y(new) = y + (k1 + 2k2 + 2k3 + k4)/6
x(new) = x + h
```

Pseudocode

1. Start
 2. Read $f(x,y)$, initial values x_0 and y_0 , step size h , and final value x_n
 3. Compute number of steps $n = (x_n - x_0)/h$
 4. Repeat n times: compute k_1, k_2, k_3, k_4 using the formulas above
 5. Update $y = y + (k_1 + 2k_2 + 2k_3 + k_4)/6$ and $x = x + h$
 6. Display the current x and y
 7. Repeat step 4 until x reaches x_n
 8. Stop
- 4.3 Flowchart

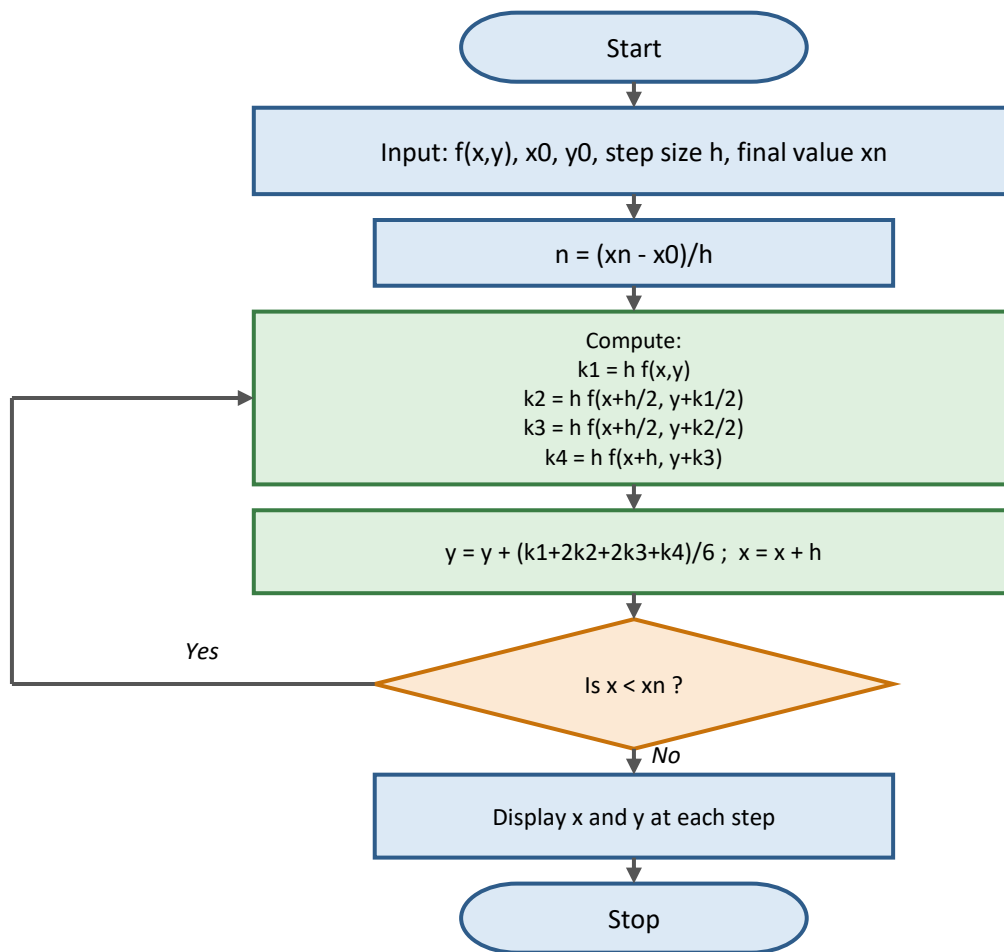


Figure 3: Flowchart of the 4th-Order Runge-Kutta Method

MATLAB Code - "for" loop version

Where $n = (x_n - x_0)/h$

```

clc; clear;
f = @(x,y) x + y;      % dy/dx = f(x,y)

x0 = 0; y0 = 1;        % initial condition
h = 0.1;               % step size
xn = 0.4;              % point at which y is required
n = (xn - x0)/h;       % number of steps

x = x0; y = y0;
for i = 1:n
    k1 = h*f(x, y);
    k2 = h*f(x + h/2, y + k1/2);
    k3 = h*f(x + h/2, y + k2/2);
    k4 = h*f(x + h, y + k3);
    y = y + (k1 + 2*k2 + 2*k3 + k4)/6;
    x = x + h;
    fprintf('x = %.2f, y = %.6f\n', x, y);
end

fprintf('y(%.2f) = %.6f\n', xn, y);
  
```

Sample Output:

```
x = 0.10, y = 1.110342
x = 0.20, y = 1.242805
x = 0.30, y = 1.399717
x = 0.40, y = 1.583648
y(0.40) = 1.583648
```

MATLAB Code - "while" loop version

```
clc; clear;
f = @(x,y) x + y;

x0 = 0; y0 = 1;
h = 0.1;
xn = 0.4;

x = x0; y = y0;
while x < xn - 1e-10 % small tolerance avoids floating-point issues
    k1 = h*f(x, y);
    k2 = h*f(x + h/2, y + k1/2);
    k3 = h*f(x + h/2, y + k2/2);
    k4 = h*f(x + h, y + k3);
    y = y + (k1 + 2*k2 + 2*k3 + k4)/6;
    x = x + h;
    fprintf('x = %.2f, y = %.6f\n', x, y);
end

fprintf('y(%.2f) = %.6f\n', xn, y);
```

Sample Output:

```
x = 0.10, y = 1.110342
x = 0.20, y = 1.242805
x = 0.30, y = 1.399717
x = 0.40, y = 1.583648
y(0.40) = 1.583648
```

More Examples

Example 1: $dy/dx = x + y$, $y(0) = 1$, $h = 0.1$.

$S(x = 0, y = 1)$:

$$k_1 = 0.1 (0 + 1) = 0.100000$$

$$k_2 = 0.1 (0.05 + 1.05) = 0.110000$$

$$k_3 = 0.1 (0.05 + 1.055) = 0.110500$$

$$k_4 = 0.1 (0.1 + 1.1105) = 0.121050$$

$$y_1 = 1 + (0.1 + 2(0.11) + 2(0.1105) + 0.12105)/6$$

$$= 1 + 0.66205/6 = 1.110342 \quad \text{at } x = 0.1$$

All steps for Example 1:

Step	x	y	k1	k2	k3	k4	y(new)
1	0.0	1.000000	0.100000	0.110000	0.110500	0.121050	1.110342
2	0.1	1.110342	0.121034	0.132086	0.132638	0.144298	1.242805
3	0.2	1.242805	0.144281	0.156495	0.157105	0.169991	1.399717
4	0.3	1.399717	0.169972	0.183470	0.184145	0.198386	1.583648

Check: the exact solution is $y = 2e^x - x - 1$, so $y(0.4) = 1.583649$. RK4 gave 1.583648, an error of about 0.000001.

Example 2: $dy/dx = y$, $y(0) = 1$, $h = 0.1$, find $y(0.2)$.

Step	x	y	k1	k2	k3	k4	y(new)
1	0.0	1.000000	0.100000	0.105000	0.105250	0.110525	1.105171
2	0.1	1.105171	0.110517	0.116043	0.116319	0.122149	1.221403

The exact value is $e^{0.2} = 1.221403$, which matches the RK4 result 1.221403.